

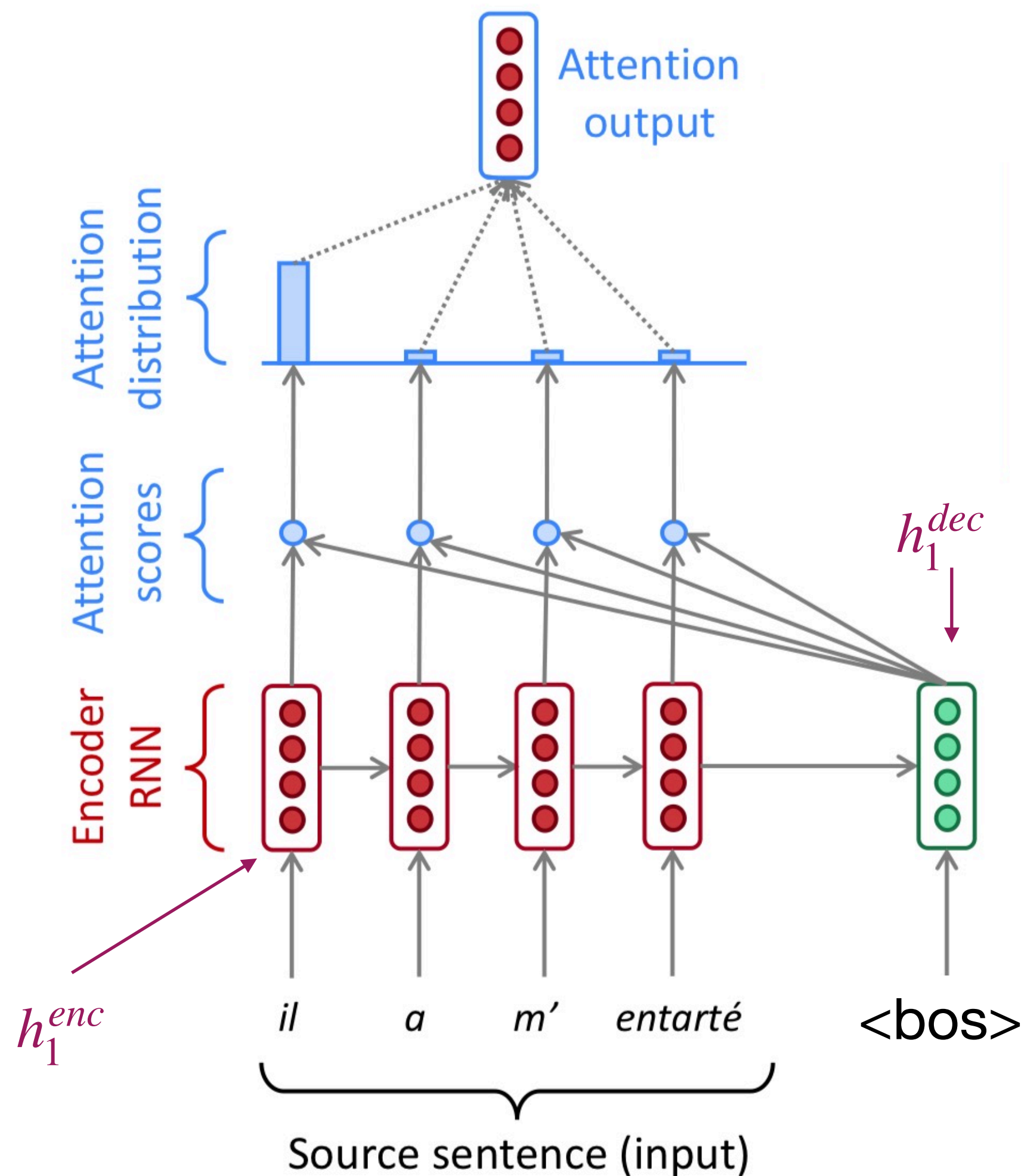
Transformers



CS 288 Spring 2026
UC Berkeley
cal-cs288.github.io/sp26

Berkeley **BAIR**
EECS

Recap: Attention



- Encoder hidden states: $h_1^{enc}, \dots, h_n^{enc}$ (n: # of words in source sentence)
- Decoder hidden state at time t : h_t^{dec}
- First, get attention scores for this time step of decoder:

$$e^t = [g(h_1^{enc}, h_t^{dec}), \dots, g(h_n^{enc}, h_t^{dec})]$$
- Obtain the attention distribution using softmax:

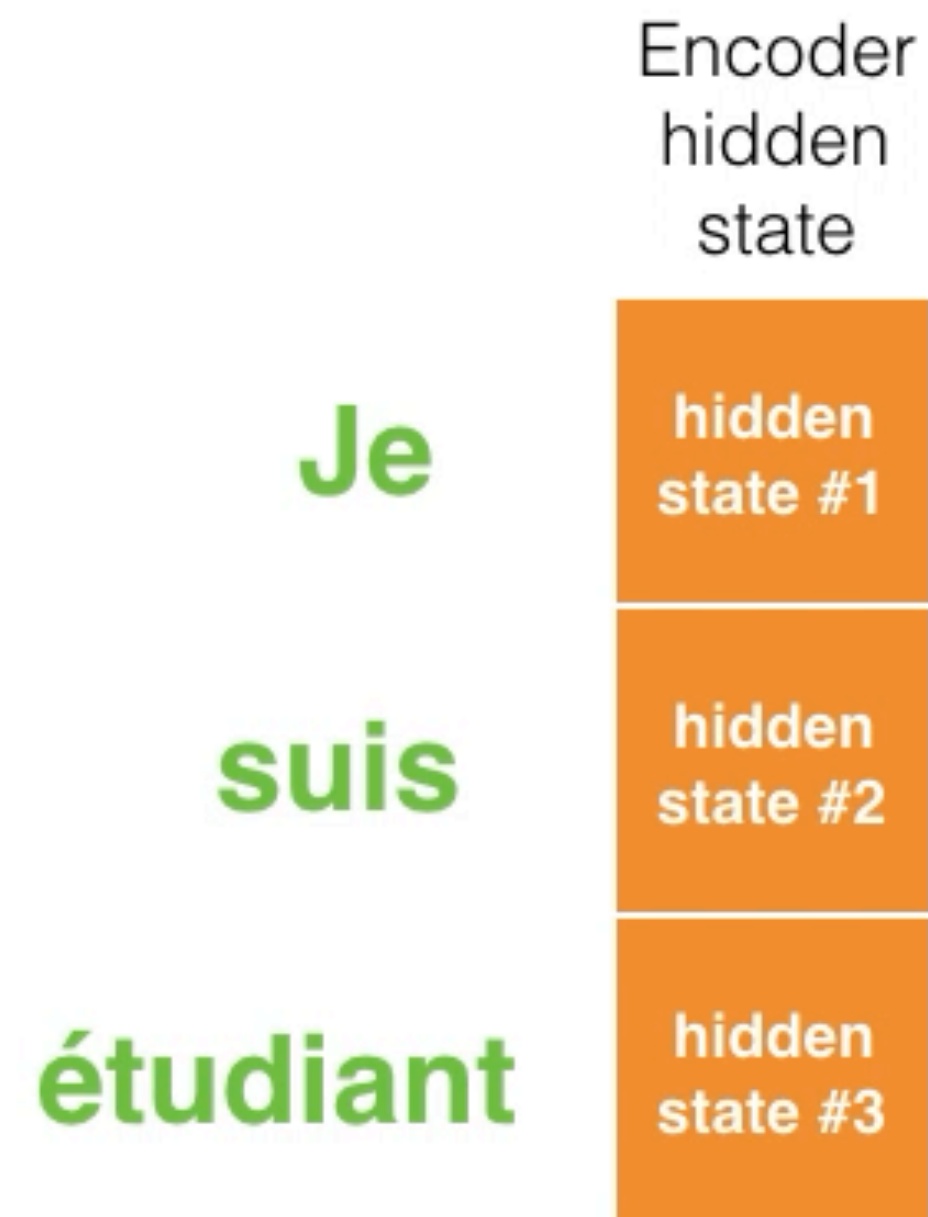
$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^n$$
- Compute weighted sum of encoder hidden states:

$$a_t = \sum_{i=1}^n \alpha_i^t h_i^{enc} \in \mathbb{R}^h$$
- Finally, concatenate with decoder state and pass on to output layer:

$$\tilde{h}_t = \tanh(\mathbf{W}_c [a_t; h_t^{dec}]) \in \mathbb{R}^h \quad \mathbf{W}_c \in \mathbb{R}^{2h \times h}$$

$$\hat{y}_t = \text{softmax}(\mathbf{W}_o \tilde{h}_t)$$

Recap: Attention

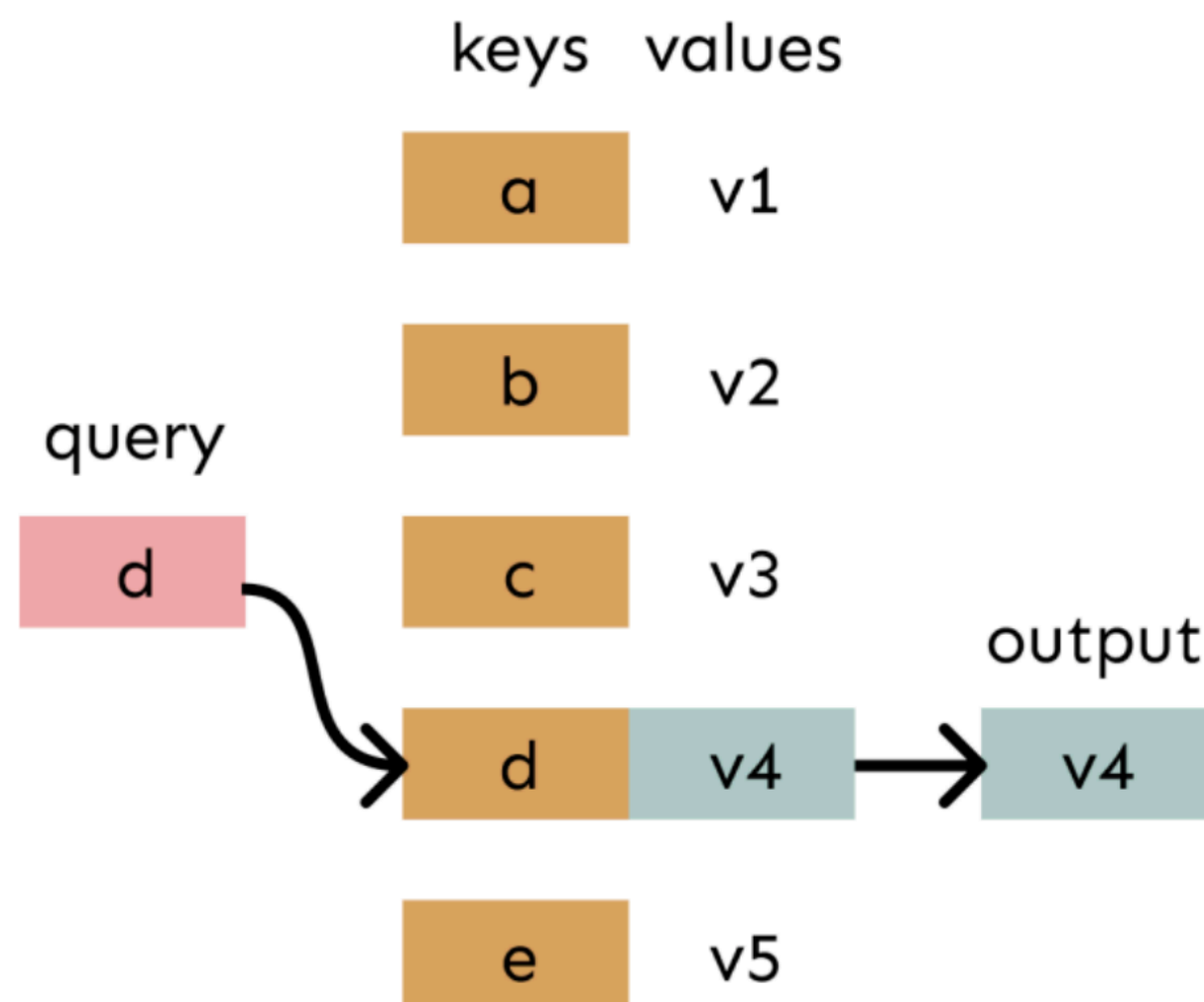


<https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

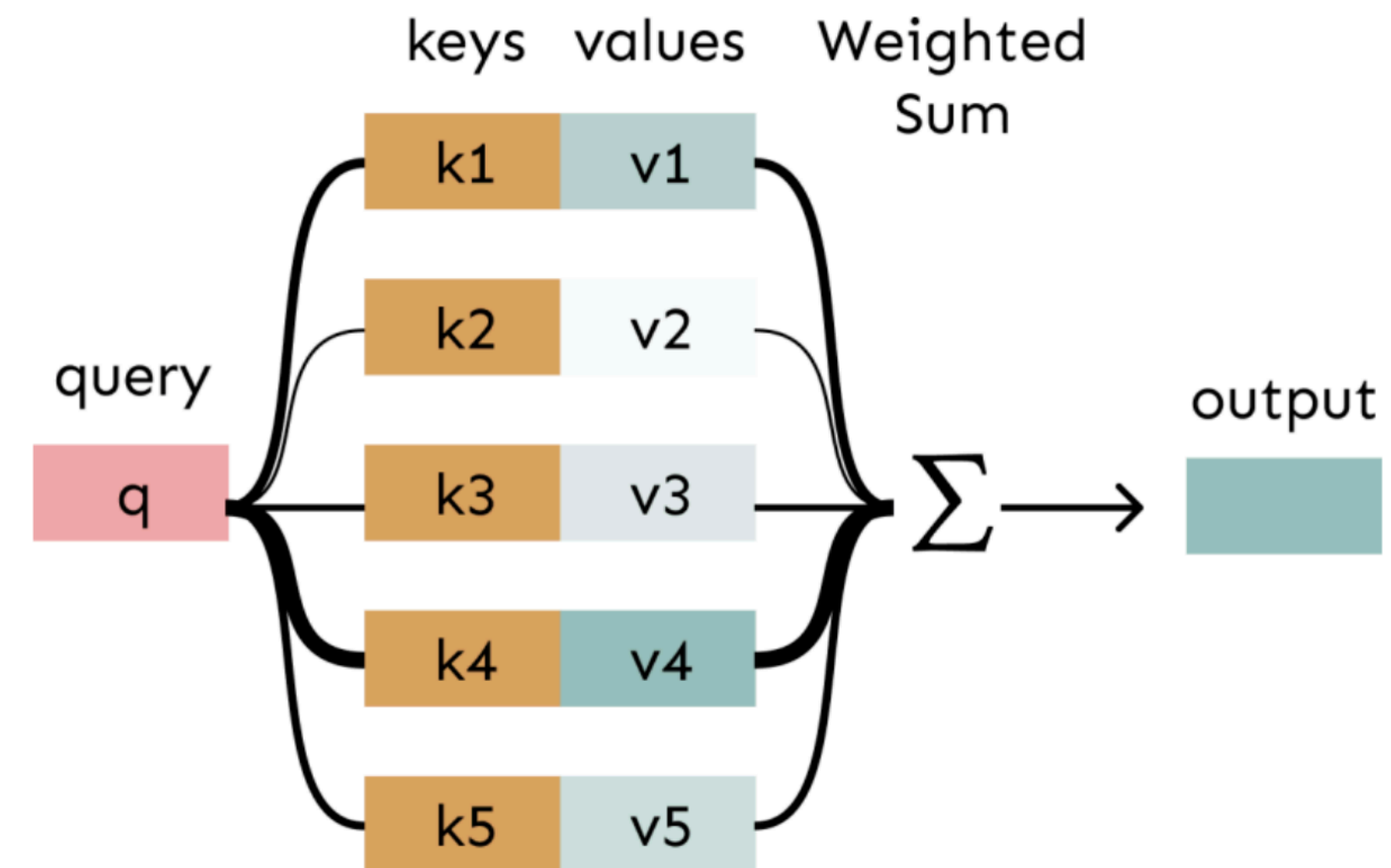
Attention as a soft, averaging lookup table

- ▶ We can think of **attention** as performing fuzzy lookup a in **key-value store**

Lookup table: a table of **keys** that map to **values**. The **query** matches one of the keys, returning its value.



Attention: The **query** matches to all **keys** softly to a weight between 0 and 1. The keys' **values** are multiplied by the weights and summed.



(So far, we assume key = value)

Transformers

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

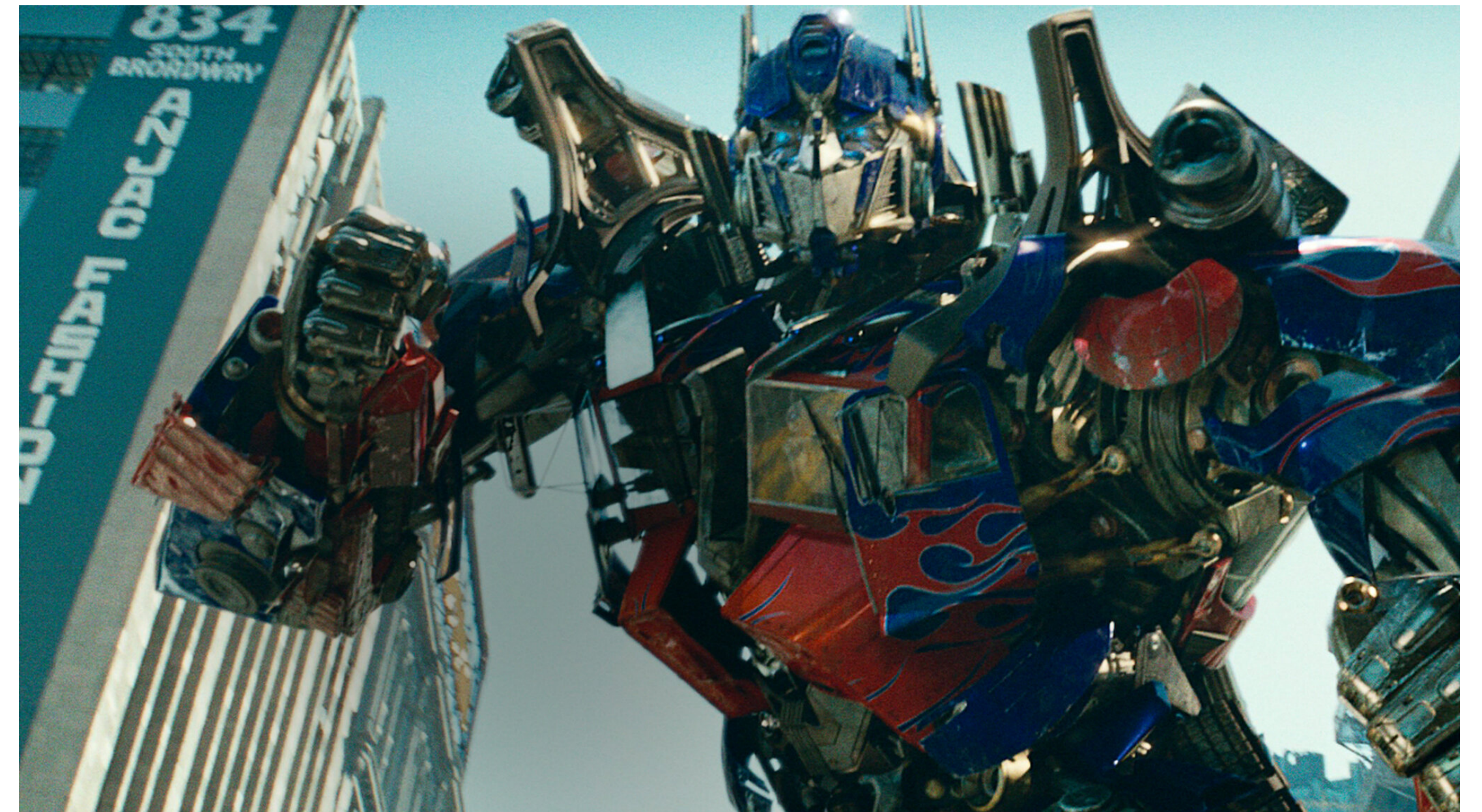
Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

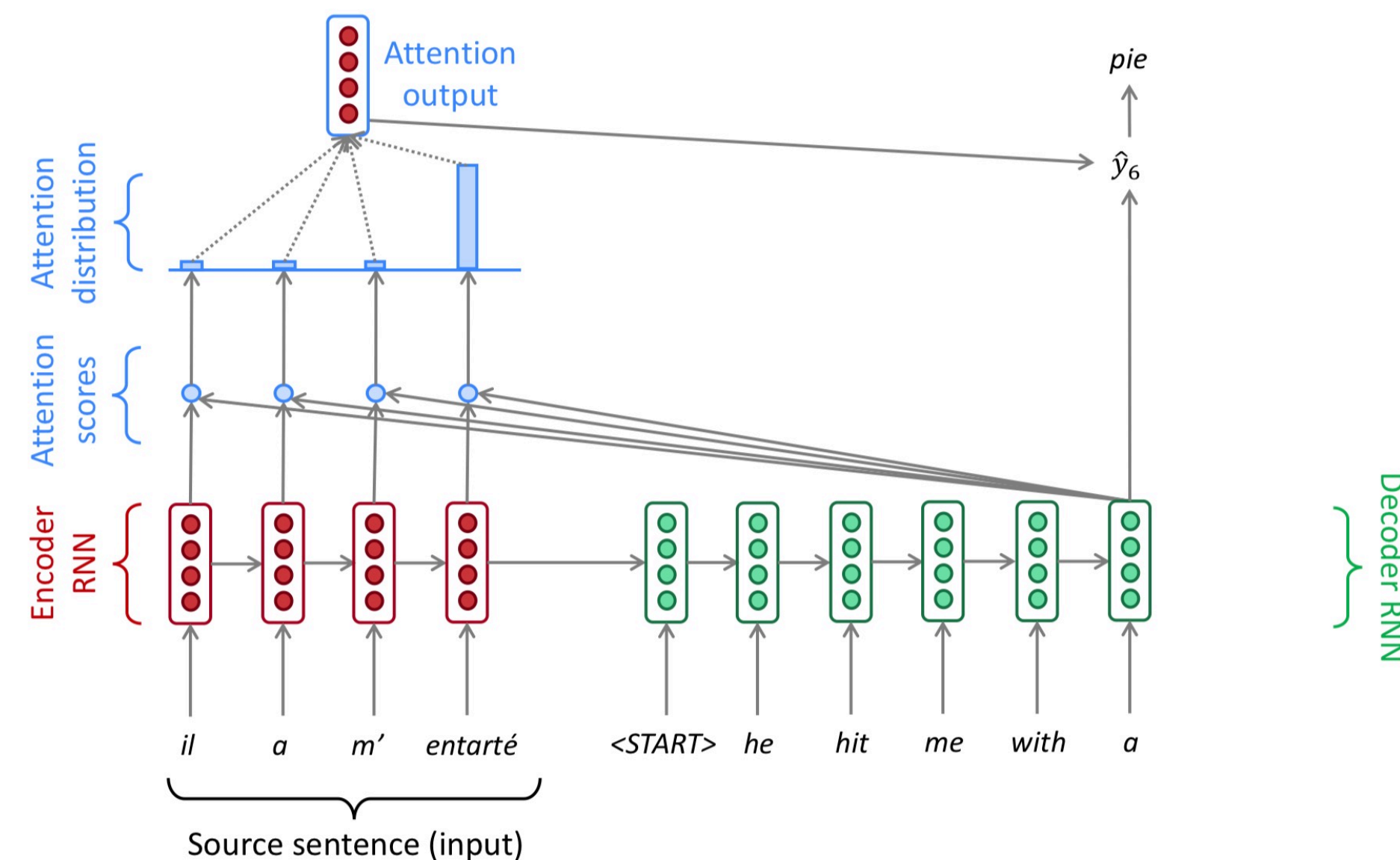
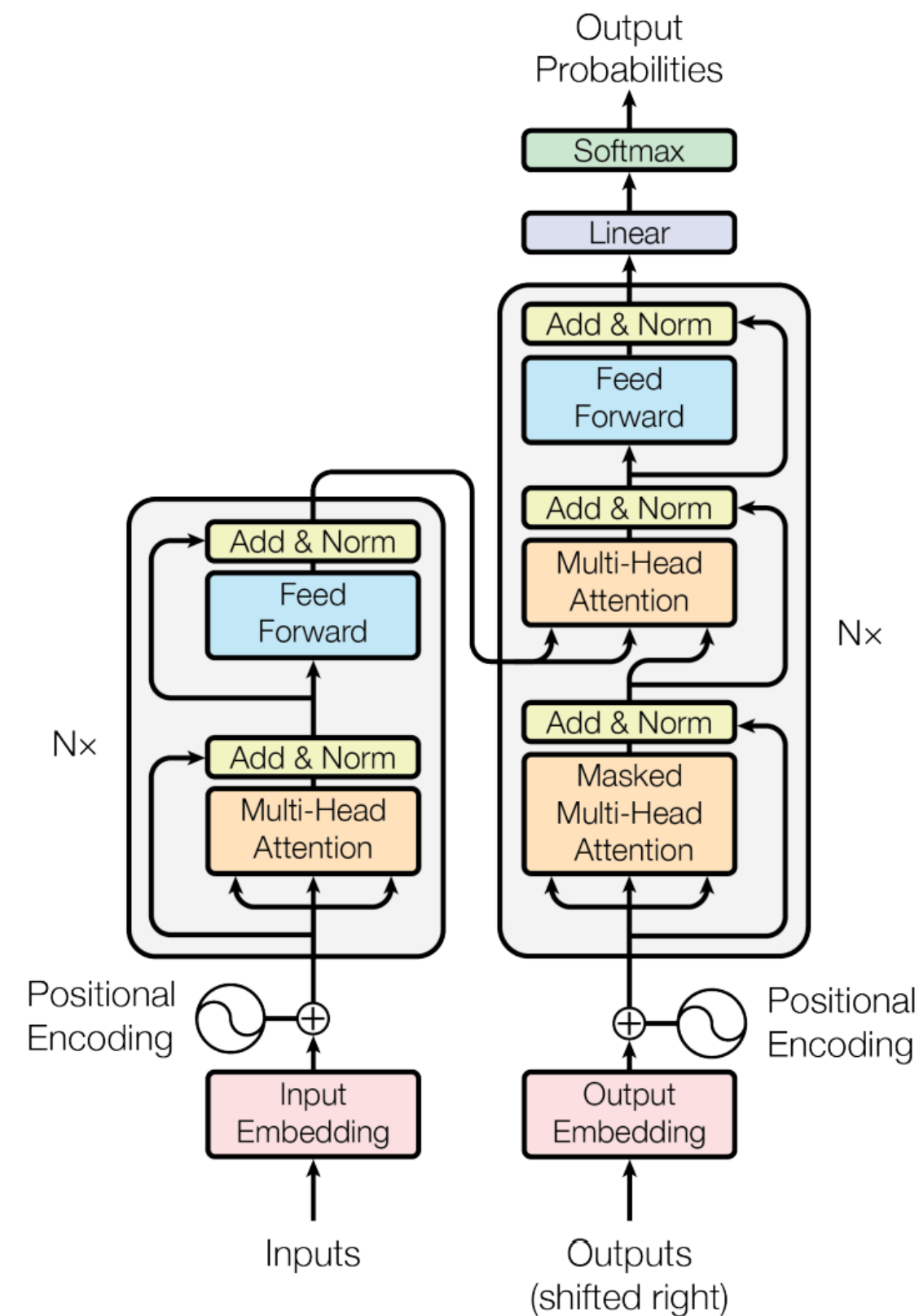
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

(Vaswani et al., 2017)

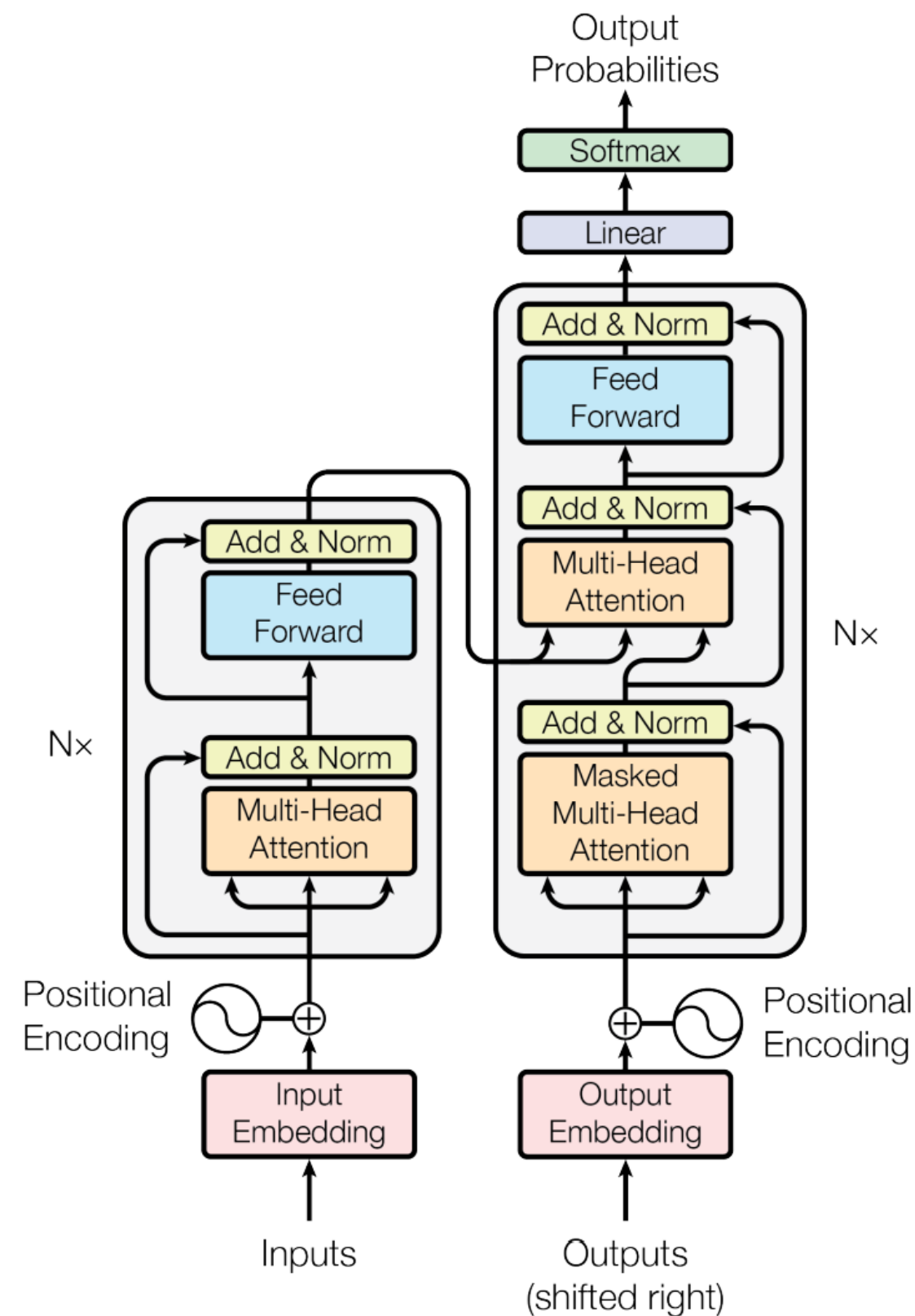


Transformer encoder-decoder

- ▶ Transformer encoder + Transformer decoder
- ▶ First designed and experimented on NMT
- ▶ Can be viewed as a replacement for seq2seq + attention based on RNNs



Transformer encoder-decoder



- ▶ Transformer encoder = a stack of **encoder layers**
- ▶ Transformer decoder = a stack of **decoder layers**

Transformer encoder: BERT, RoBERTa, ELECTRA

Transformer decoder: GPT-2, GPT-3, Llama

Transformer encoder-decoder: T5, BART

- ▶ Key innovation: **multi-head, self-attention**
- ▶ Clean & effective architecture design
- ▶ Transformers don't have any recurrence structure

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t) \in \mathbb{R}^h$$

Issues with recurrent RNNs

- ▶ Longer sequences can lead to vanishing gradients -> it's hard to capture long-distance information
- ▶ RNNs lack parallelizability
 - ▶ Forward and backward passes have $O(\text{sequence length})$ unparallelizable operations
 - ▶ GPUs can perform a bunch of independent computations at once!
 - ▶ Inhibits training on very large datasets

RNNs / LSTMs → seq2seq → seq2seq + attention → attention only = Transformers!

Transformers have become a new building block to replace RNNs

Transformers: Roadmap

1. Attention in Transformers

- ▶ From attention to self-attention
- ▶ From self-attention to multi-head self-attention
- ▶ From multi-head self-attention to masked multi-head self-attention

2. Zooming out: Transformers

- ▶ Embeddings and positional encoding
- ▶ Feedforward layers
- ▶ Residual connections + layer normalization
- ▶ Transformer encoder vs Transformer decoder vs. Transformer encoder-decoder

3. Modern variants of Transformers

- ▶ Embeddings and positional encoding
- ▶ Feedforward layers
- ▶ Pre-norm vs. Post-norm
- ▶ Layernorm vs. RMSNorm
- ▶ Activations
- ▶ Positional Encodings
- ▶ Attention

Transformers: Roadmap

1. Attention in Transformers

- ▶ From attention to self-attention
- ▶ From self-attention to multi-head self-attention
- ▶ From multi-head self-attention to masked multi-head self-attention

2. Zooming out: Transformers

- ▶ Embeddings and positional encoding
- ▶ Feedforward layers
- ▶ Residual connections + layer normalization
- ▶ Transformer encoder vs Transformer decoder vs. Transformer encoder-decoder

3. Modern variants of Transformers

- ▶ Embeddings and positional encoding
- ▶ Feedforward layers
- ▶ Pre-norm vs. Post-norm
- ▶ Layernorm vs. RMSNorm
- ▶ Activations
- ▶ Positional Encodings
- ▶ Attention

Multi-head Self-Attention

Attention in a general form

- Assume that we have a set of values $\mathbf{v}_1, \dots, \mathbf{v}_n \in \mathbb{R}^{d_v}$ and a query vector $\mathbf{q} \in \mathbb{R}^{d_q}$
- Attention always involves the following steps:

- Computing the **attention scores** $\mathbf{e} = g(\mathbf{q}, \mathbf{v}_i) \in \mathbb{R}^n$
- Taking softmax to get **attention distribution** α

$$\alpha = \text{softmax}(\mathbf{e}) \in \mathbb{R}^n$$

- Using attention distribution to take **weighted sum** of values:

$$\mathbf{a} = \sum_{i=1}^n \alpha_i \mathbf{v}_i \in \mathbb{R}^{d_v}$$

Attention in a general form

- A model general form: use a set of keys and values
 $(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_n, \mathbf{v}_n), \mathbf{k}_i \in \mathbb{R}^{d_k}, \mathbf{v}_i \in \mathbb{R}^{d_v}$ where keys are used to compute the attention scores and values are used to compute the output vector

- Attention always involves the following steps:

- Computing the **attention scores** $\mathbf{e} = g(\mathbf{q}, \mathbf{k}_i) \in \mathbb{R}^n$

- Taking softmax to get **attention distribution** α

$$\alpha = \text{softmax}(\mathbf{e}) \in \mathbb{R}^n$$

- Using attention distribution to take **weighted sum** of values:

$$\mathbf{a} = \sum_{i=1}^n \alpha_i \mathbf{v}_i \in \mathbb{R}^{d_v}$$

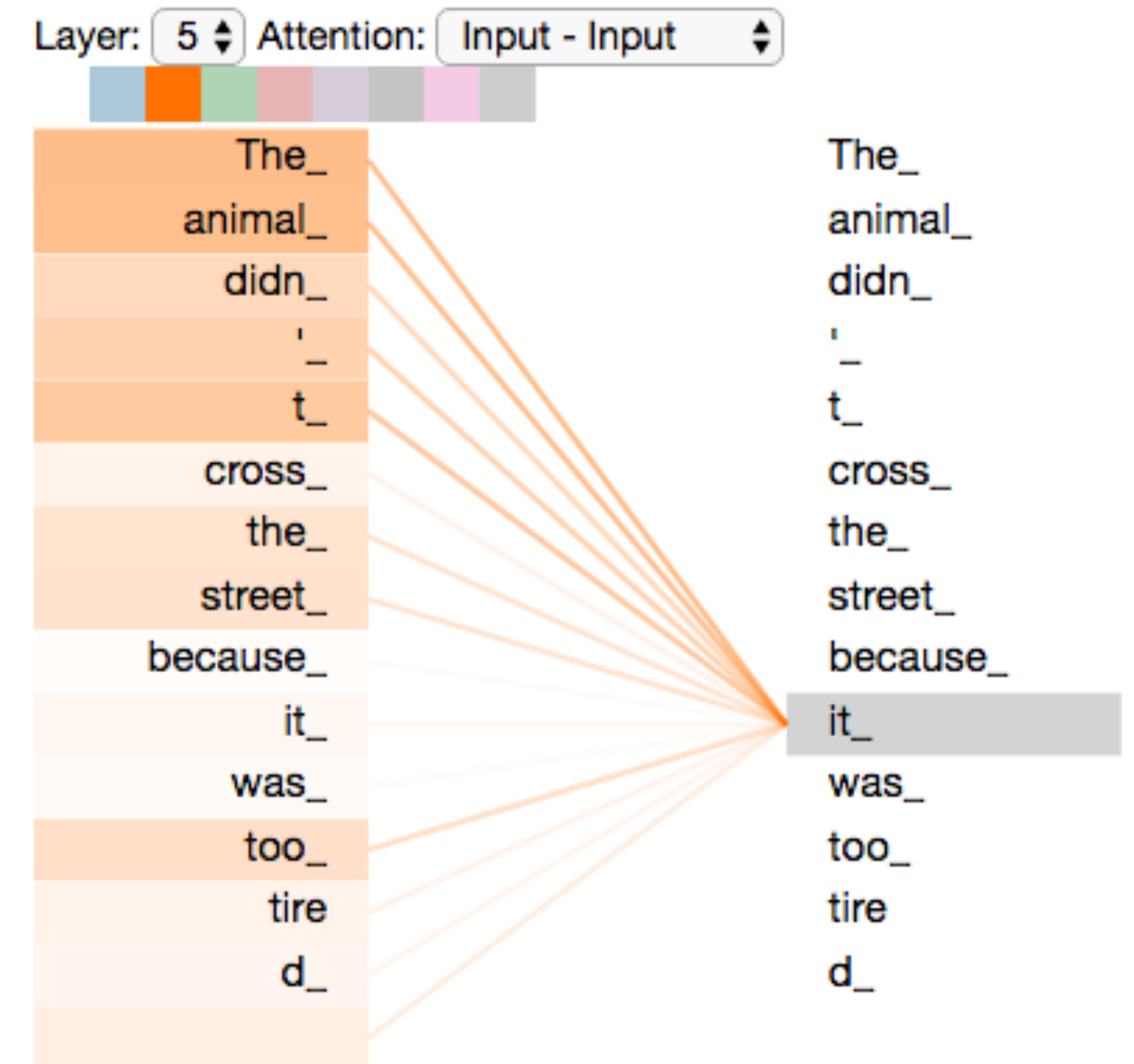
Query: "Here is what I'm interested in"

Key: "Here is what I have"

Value: "If you find me interesting, here is what I will communicate to you"

Self-attention

- In NMT, query = decoder hidden state, keys = values = encoder hidden states
- Self-attention: Attention from the sequence to itself
- Self-attention: let's use each word in a sequence as the query, and all other words in the sequence as keys and values



Self-attention

- A self attention layer maps a sequence of input vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^{d_1}$ to a sequence of n vectors $\mathbf{z}_1, \dots, \mathbf{z}_n \in \mathbb{R}^{d_2}$
- The same abstraction as RNNs - use as a drop-in replacement for an RNN layer
 $\mathbf{z}_t = g(\mathbf{W}\mathbf{z}_{t-1} + \mathbf{U}\mathbf{x}_t + \mathbf{b}) \in \mathbb{R}^{h_2}$

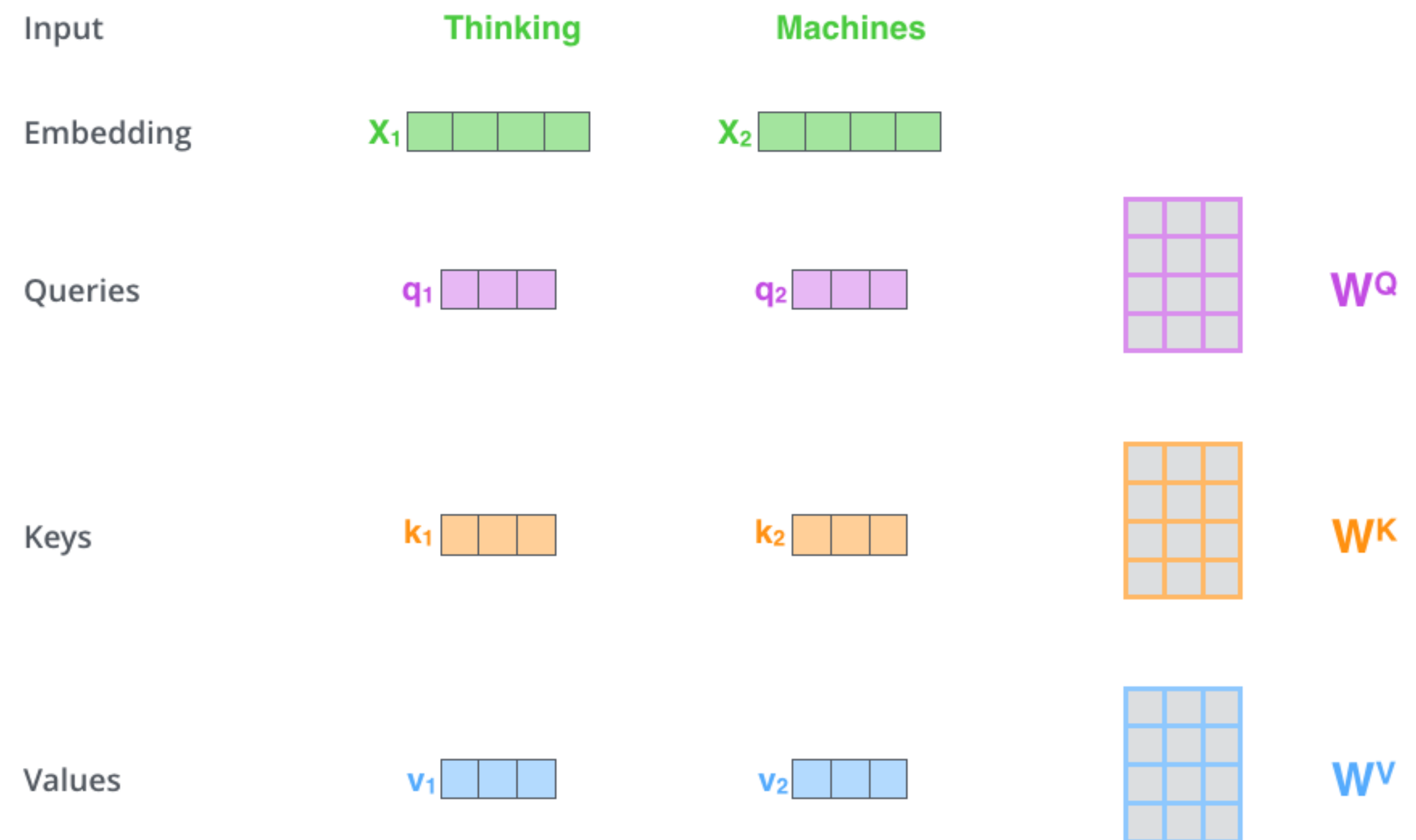
Self-attention

- Step 1: Transform each input vector into three vectors: query, key, and value vectors

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \in \mathbb{R}^{d_q}, \quad \mathbf{W}^Q \in \mathbb{R}^{d_1 \times d_q}$$

$$\mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K \in \mathbb{R}^{d_k}, \quad \mathbf{W}^K \in \mathbb{R}^{d_1 \times d_k}$$

$$\mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V \in \mathbb{R}^{d_v}, \quad \mathbf{W}^V \in \mathbb{R}^{d_1 \times d_v}$$



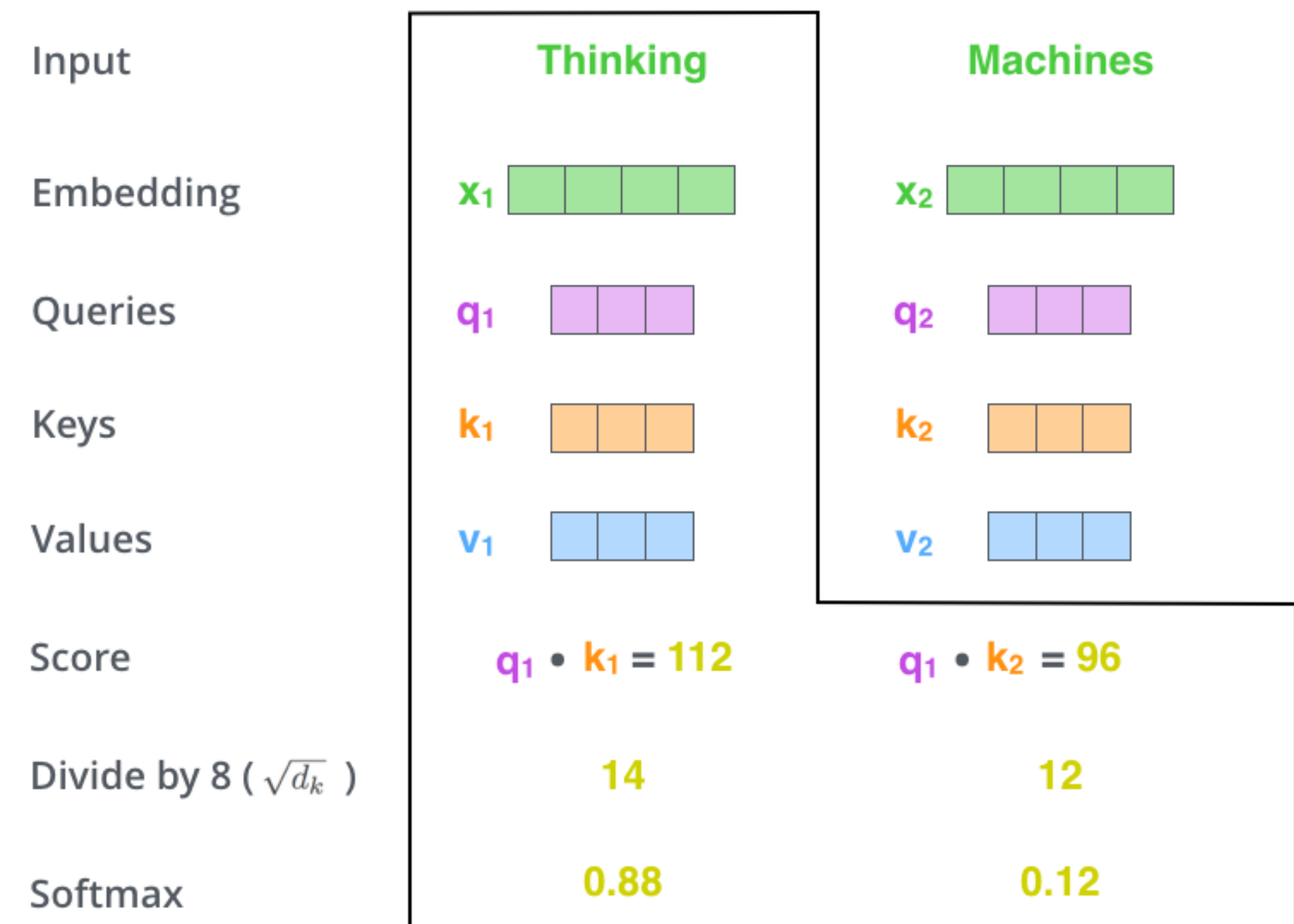
Self-attention

- Step 2: For each query-key pair, calculate the weight based on a dot product.

- Normalize to add to one using softmax.

$$\alpha_{i,j} = \text{softmax} \left(q_i \cdot k_j \right)$$

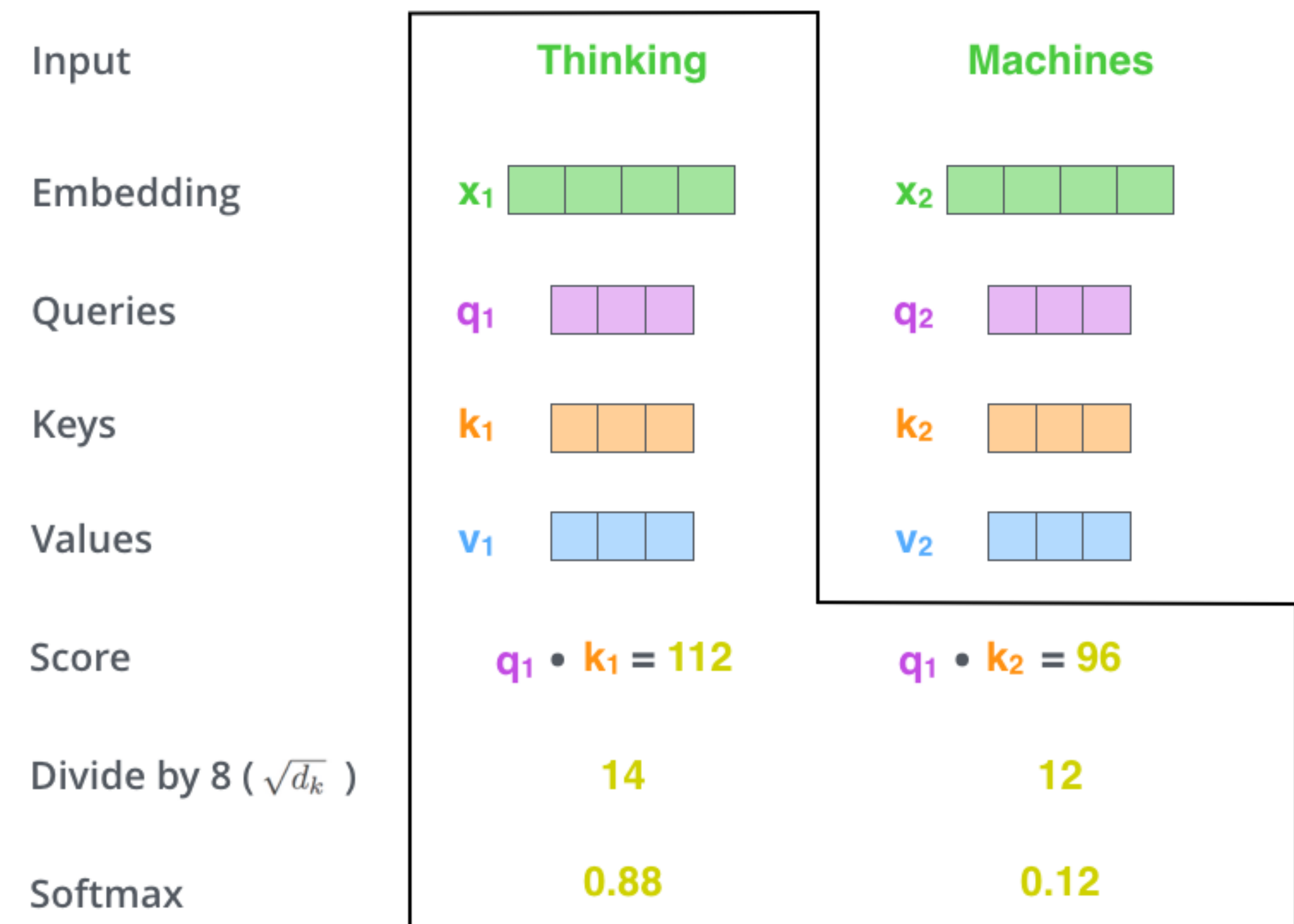
- Problem: scale of dot product increases as dimensions get larger



Self-attention

- Step 2: For each query-key pair, calculate the weight based on a dot product.
- Normalize to add to one using softmax.
- Problem: scale of dot product increases as dimensions get larger
- Fix: Scale by size of the vector

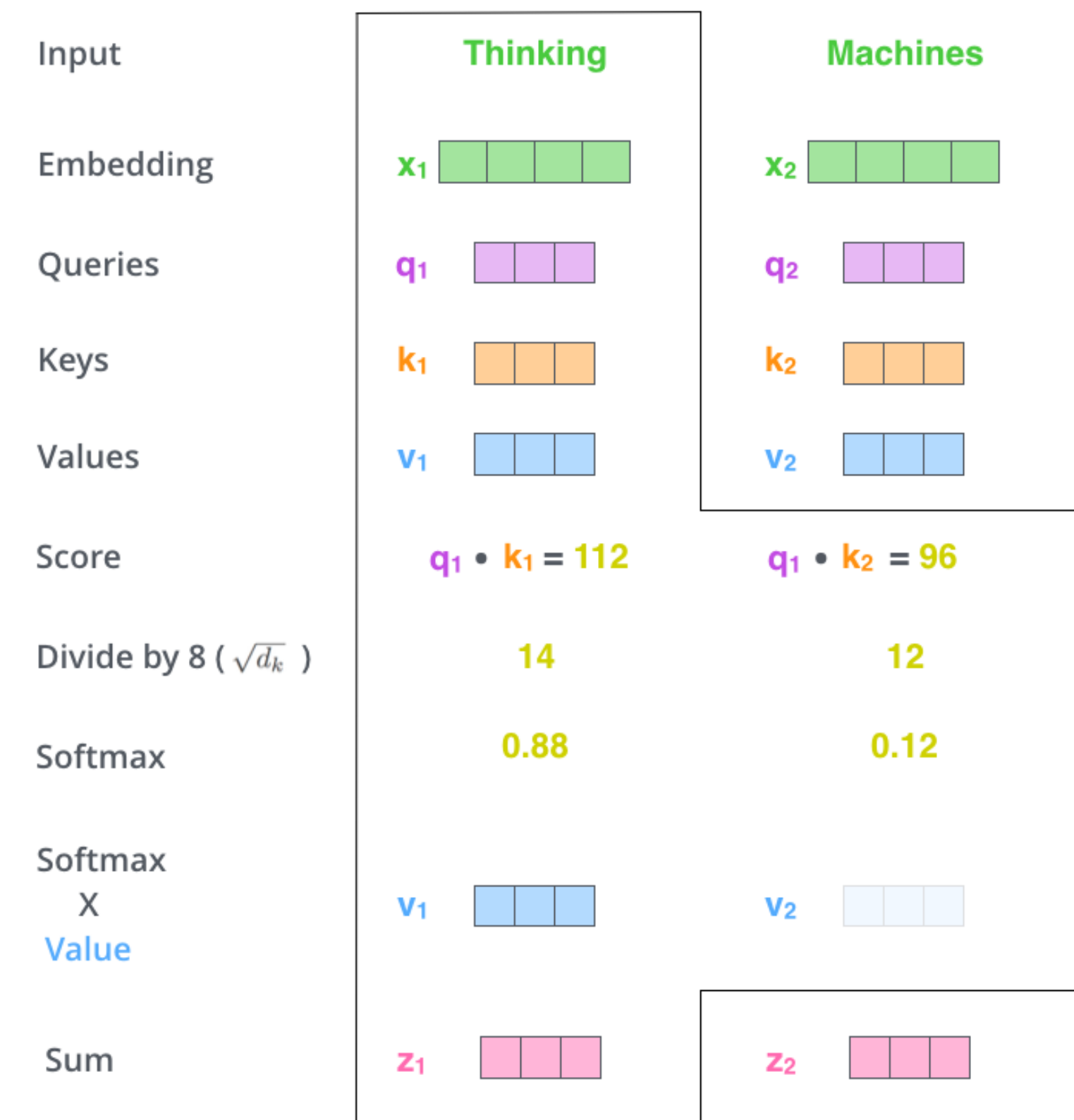
$$\alpha_{i,j} = \text{softmax} \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right)$$



Self-attention

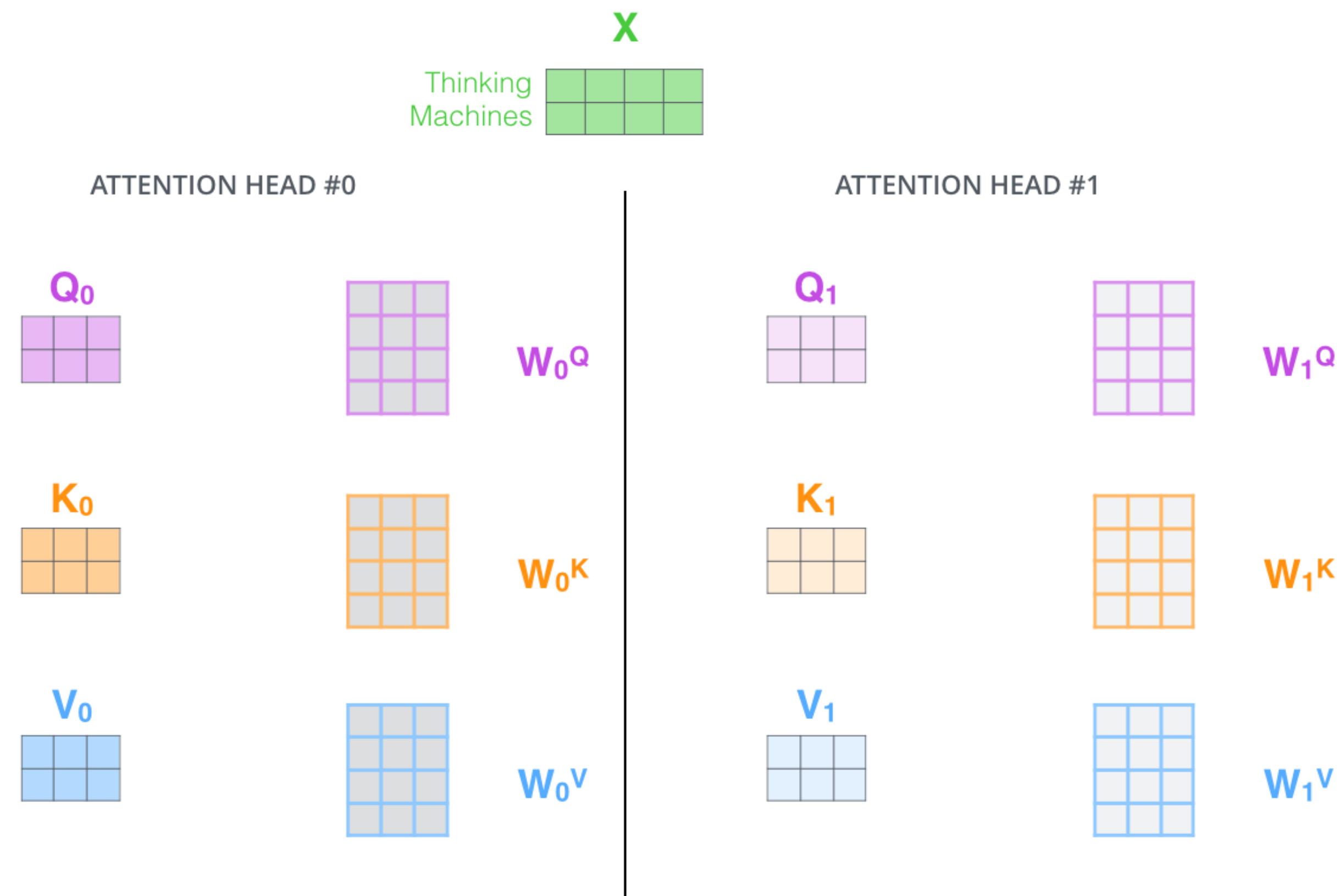
- Step 3: Compute output for each input as weighted some of values

$$\mathbf{z}_i = \sum_{j=1}^n \alpha_{i,j} \mathbf{v}_j \in \mathbb{R}^{d_v}$$



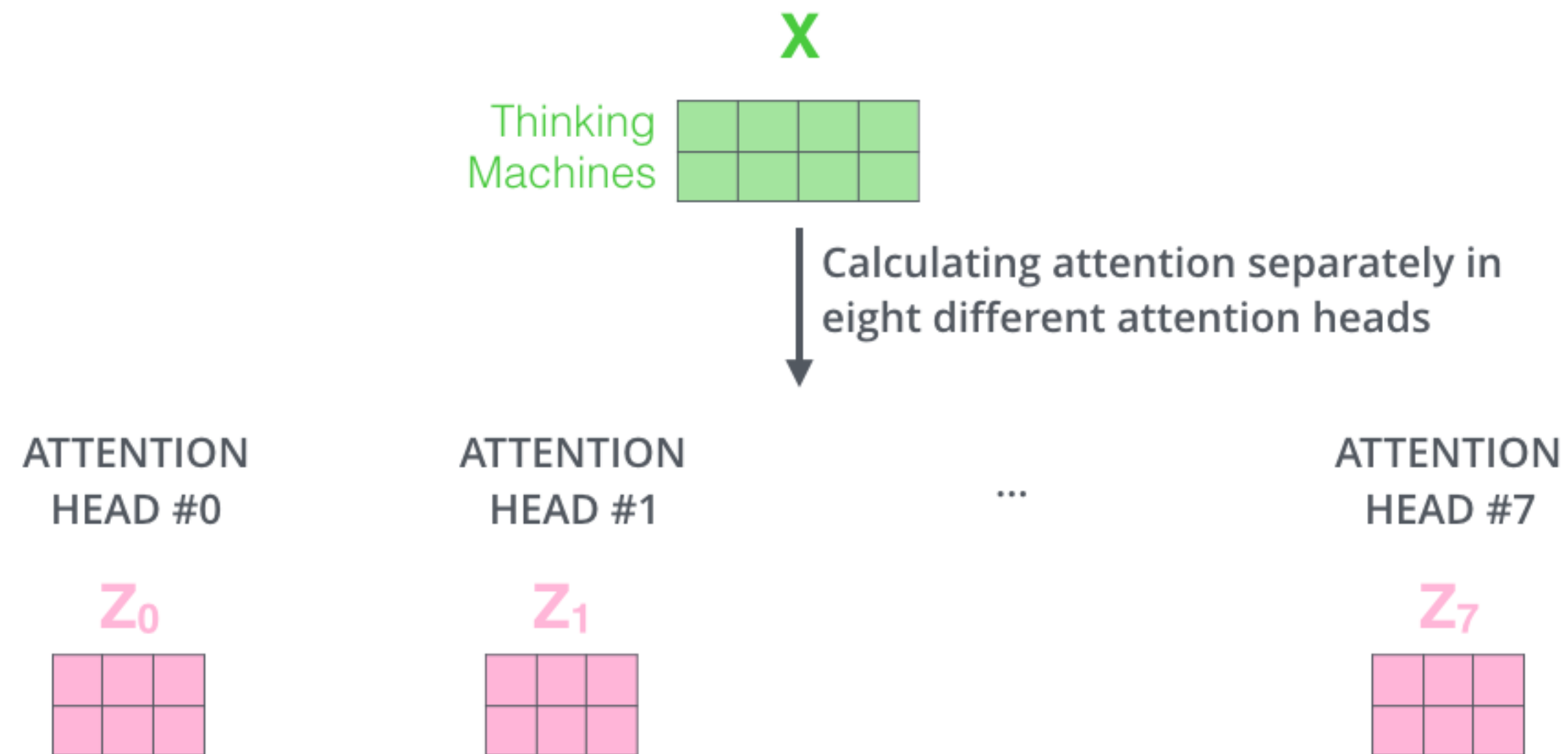
Multi-head attention

- Intuition: It's better to use multiple attention functions instead of one!
- Each attention function ("head") focuses on different key positions / content.



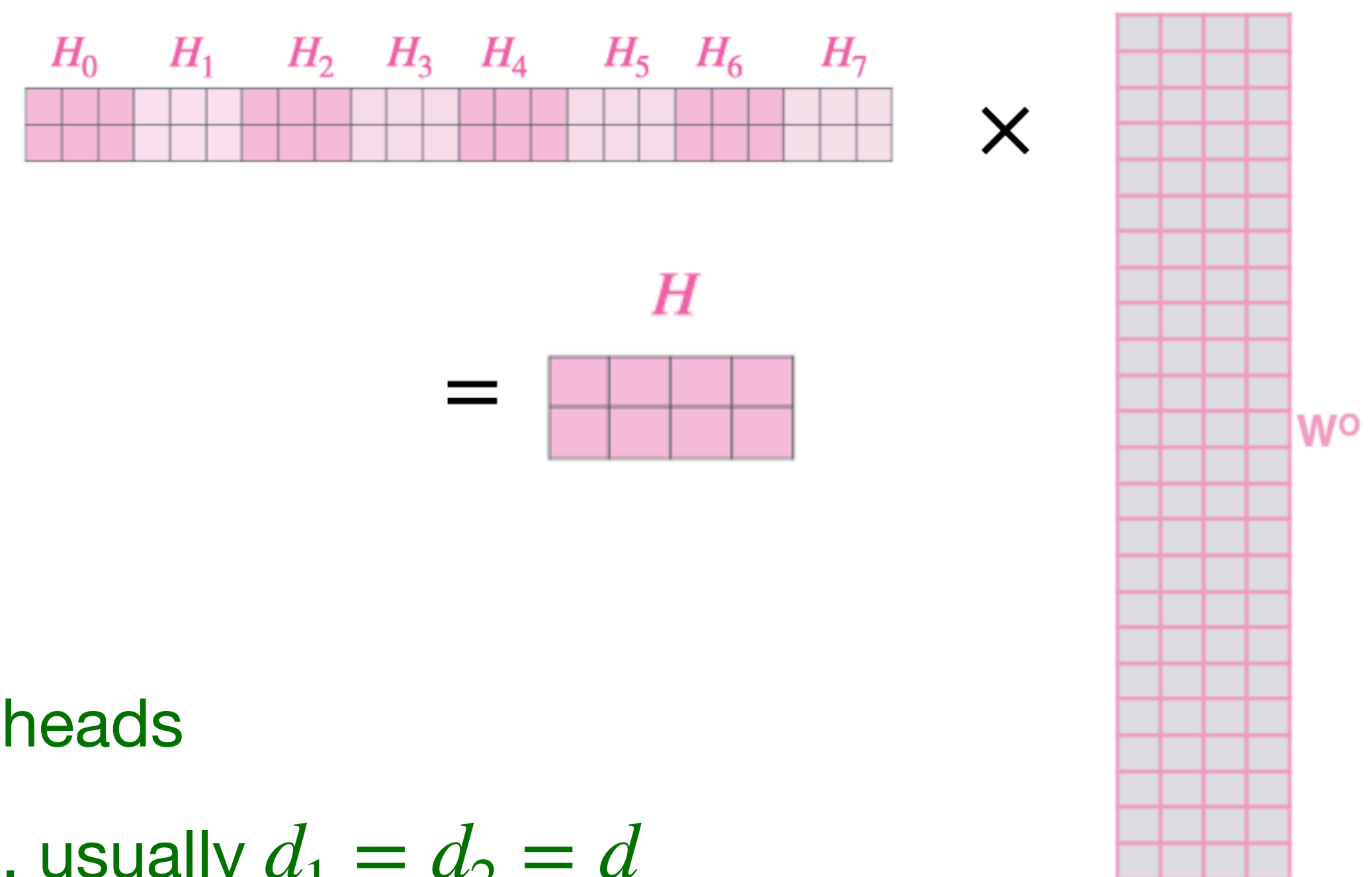
Multi-head attention

- Intuition: It's better to use multiple attention functions instead of one!
- Each attention function (“head”) focuses on different key positions / content.



Multi-head attention

- This leaves us with a bit of a challenge. The feed-forward layer is not expecting eight matrices – it's expecting a single matrix (a vector for each word). So we need a way to condense these eight down into a single matrix.
- How do we do that? We concat the matrices then multiply them by an additional weights matrix $W^O \in \mathbb{R}^{d \times d_2}$.



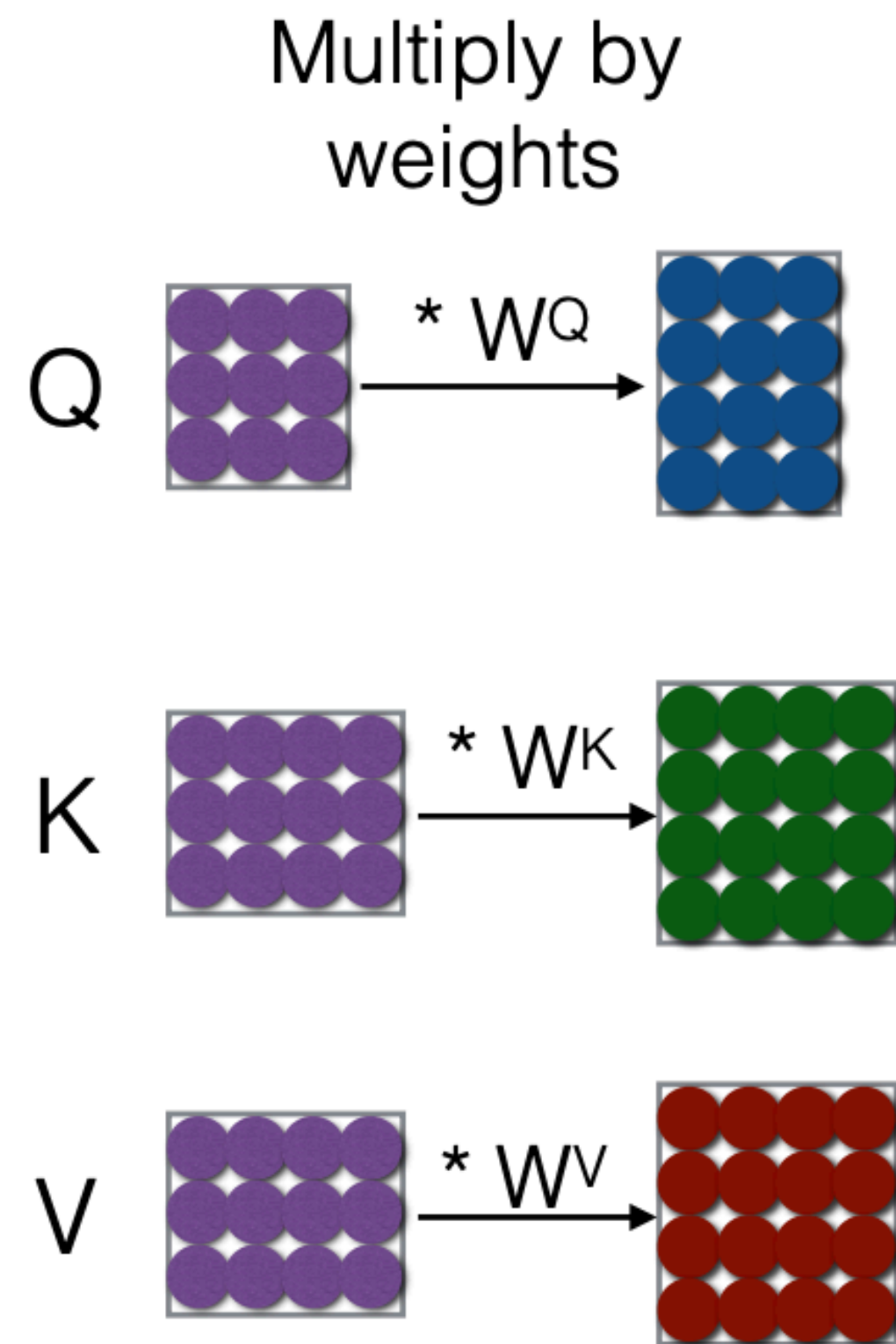
$$\mathbf{W}^Q \in \mathbb{R}^{d_1 \times d_q}, \mathbf{W}^K \in \mathbb{R}^{d_1 \times d_k}, \mathbf{W}^V \in \mathbb{R}^{d_1 \times d_v}$$

$$d_q = d_k = d_v = d/m$$

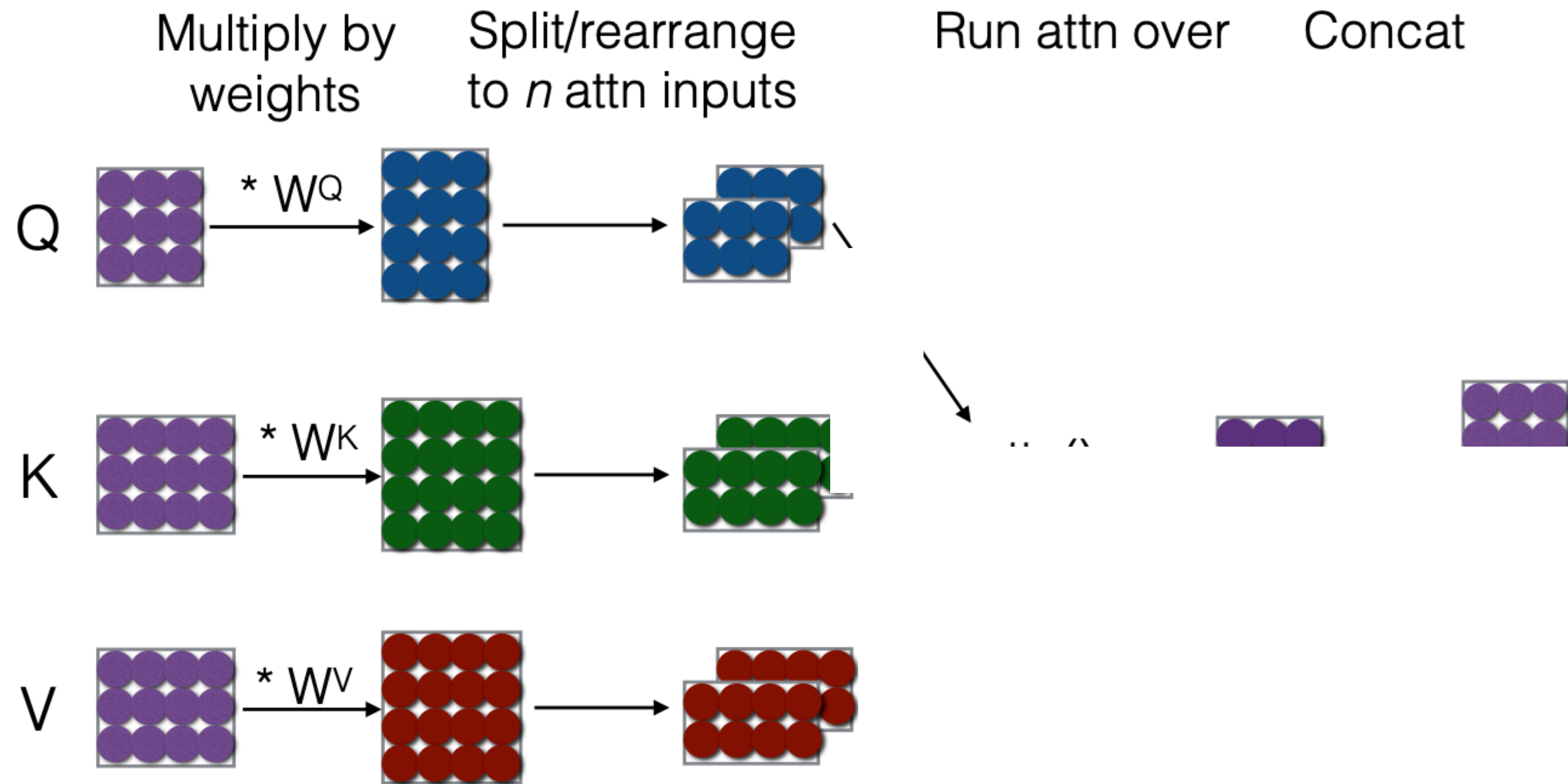
$$W^O \in \mathbb{R}^{d \times d_2}$$

- d is hidden size, m = # of heads
- If we stack multiple layers, usually $d_1 = d_2 = d$

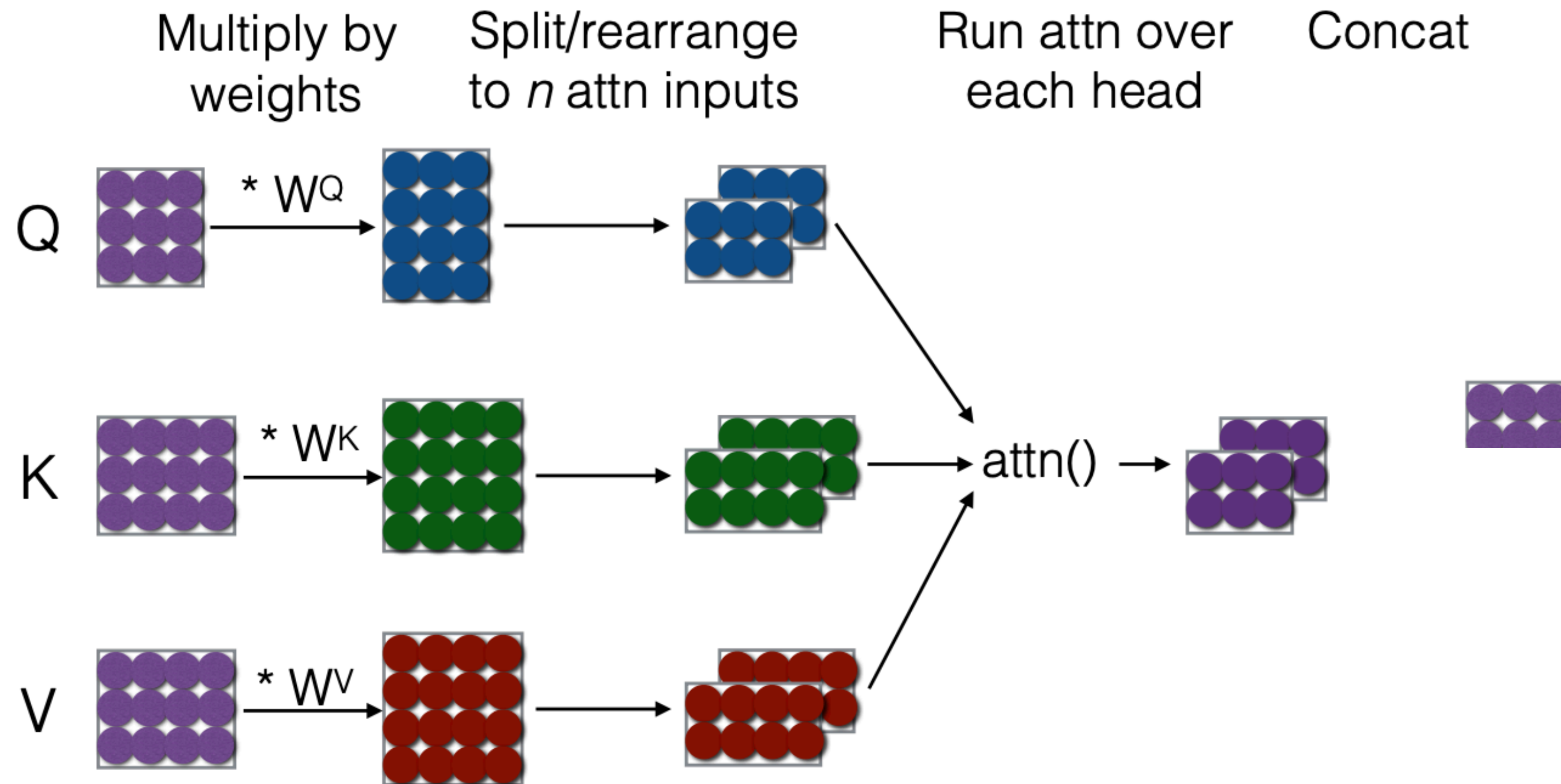
Visualization: Multi-head attention



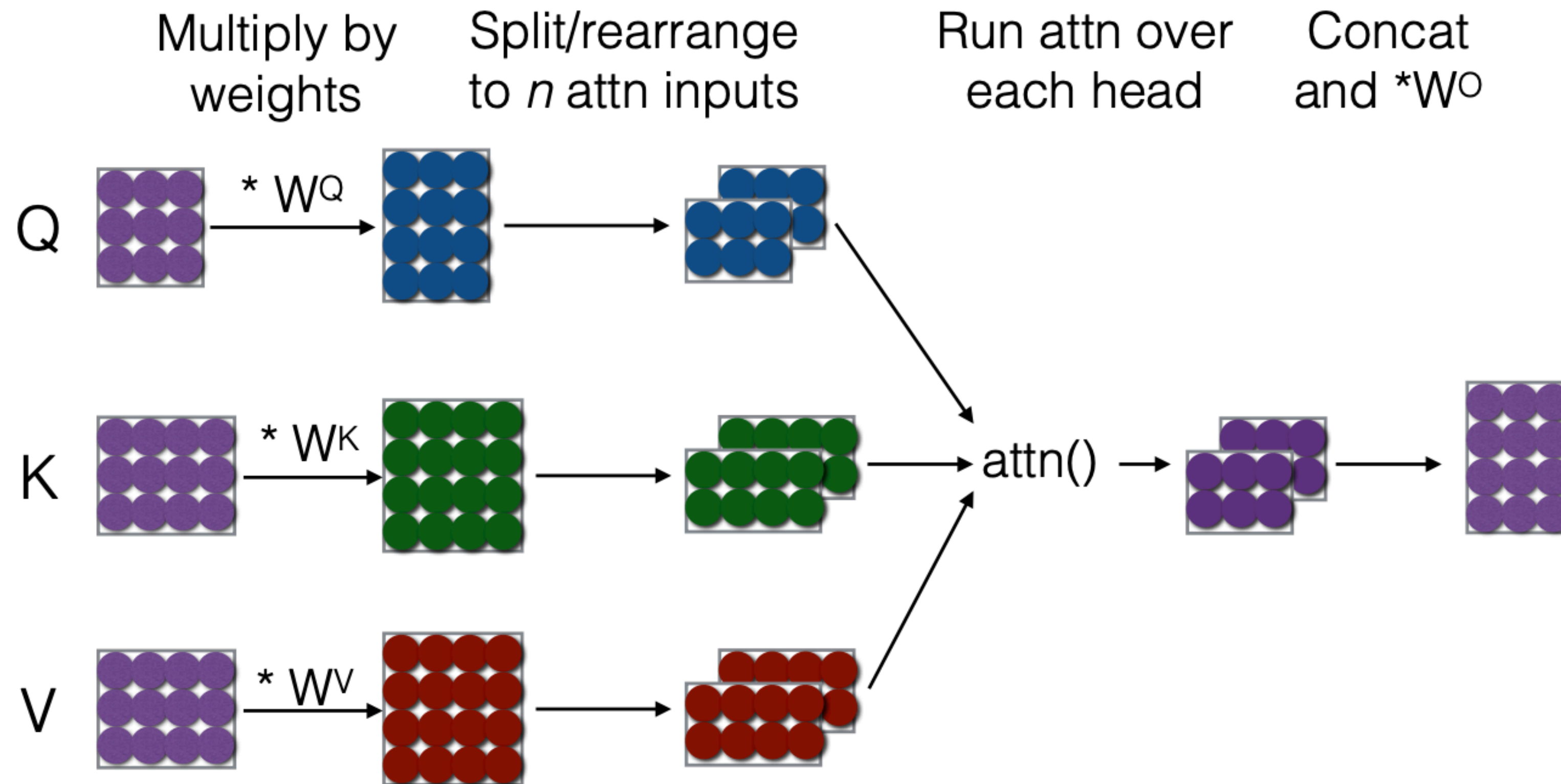
Visualization: Multi-head attention



Visualization: Multi-head attention



Visualization: Multi-head attention



Missing piece: Positional encoding

- Unlike RNNs, self-attention doesn't build in order information, we need to encode the order of the sentence in our keys, queries, and values
- Solution: Add “**positional encoding**” to the input embeddings: $\mathbf{p}_i \in \mathbb{R}^d$ for $i = 1, 2, \dots, n$

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{p}_i$$

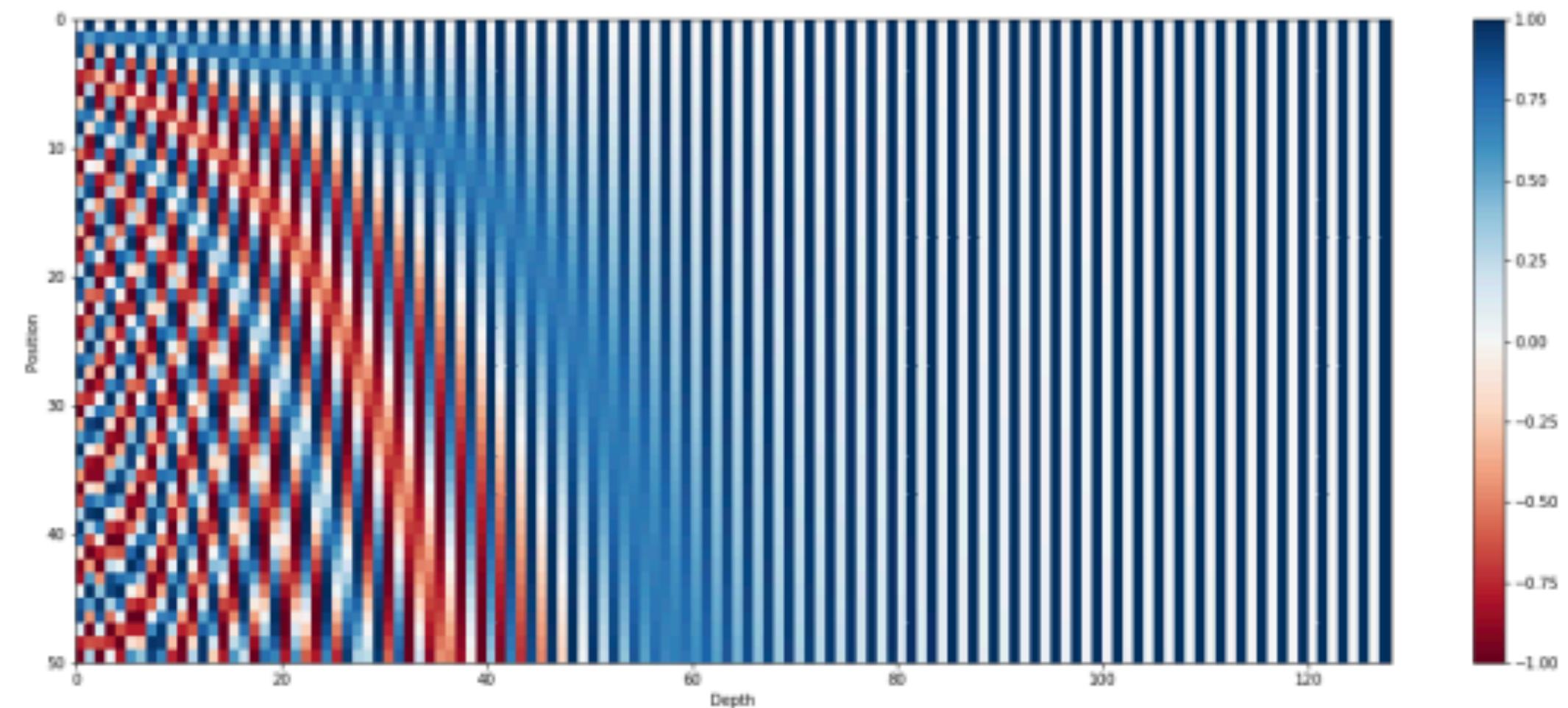
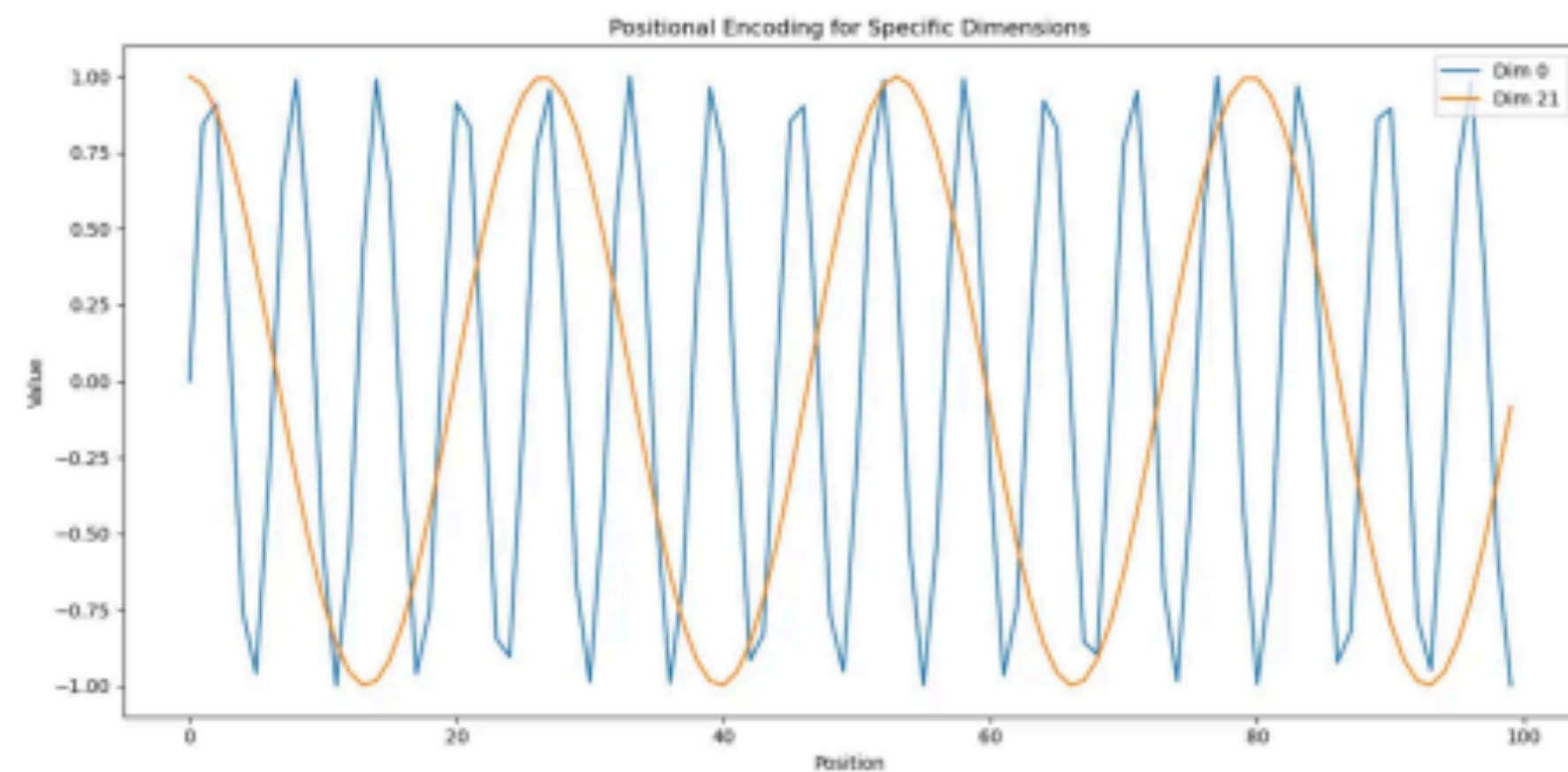
- Positional encodings add an embedding based on the word position
- Sinusoidal Encoding

$$p_{t,2i} = \sin(t/10000^{2i/h})$$
$$p_{t,2i+1} = \cos(t/10000^{2i/h})$$

Positional Encoding

Sinusoidal Encoding (Vaswani+ 2017, Kazemnejad 2019)

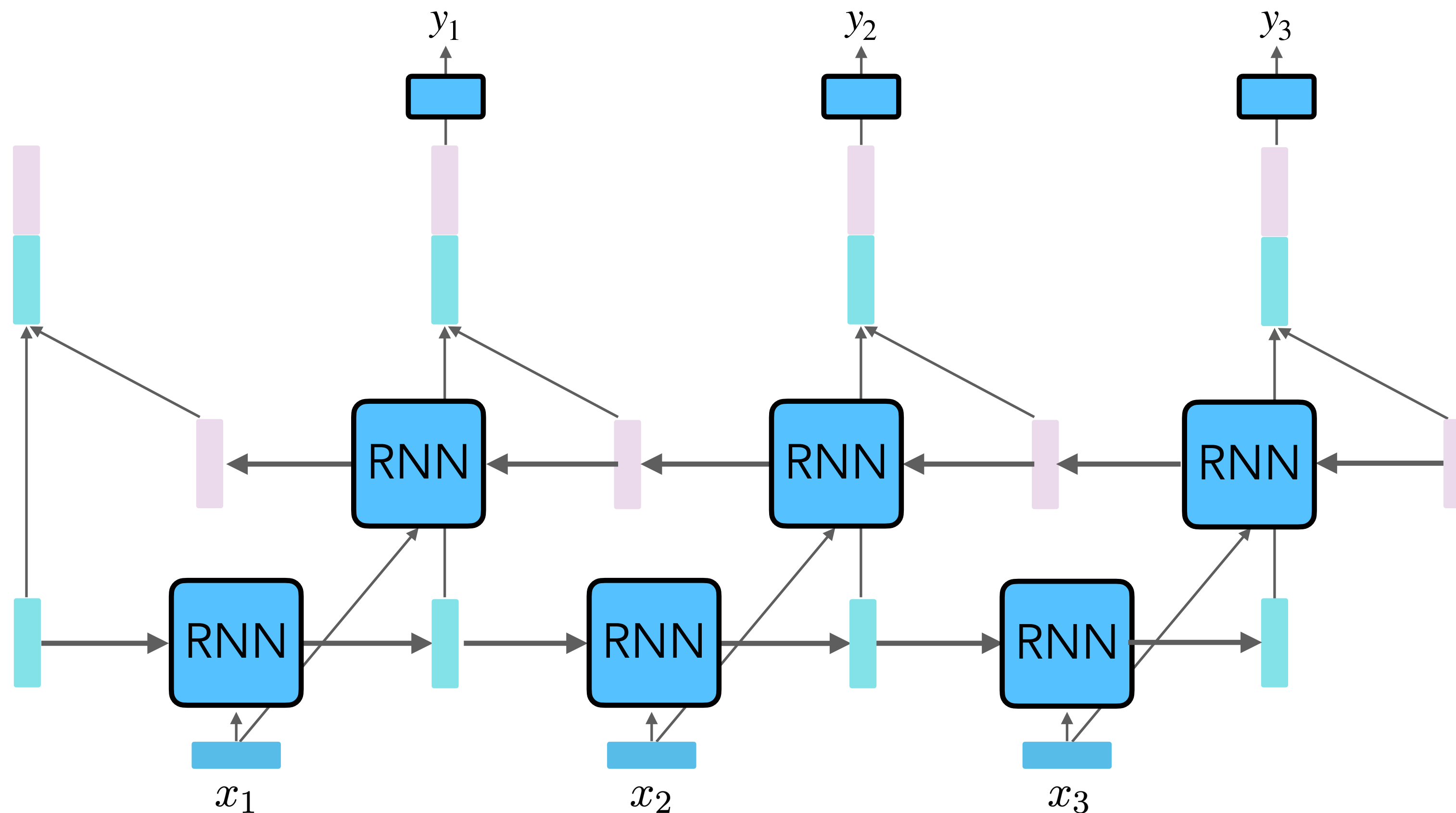
$$p_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases} \quad \text{where} \quad \omega_k = \frac{1}{10000^{2k/d}}$$



Masked Multi-head Self-Attention

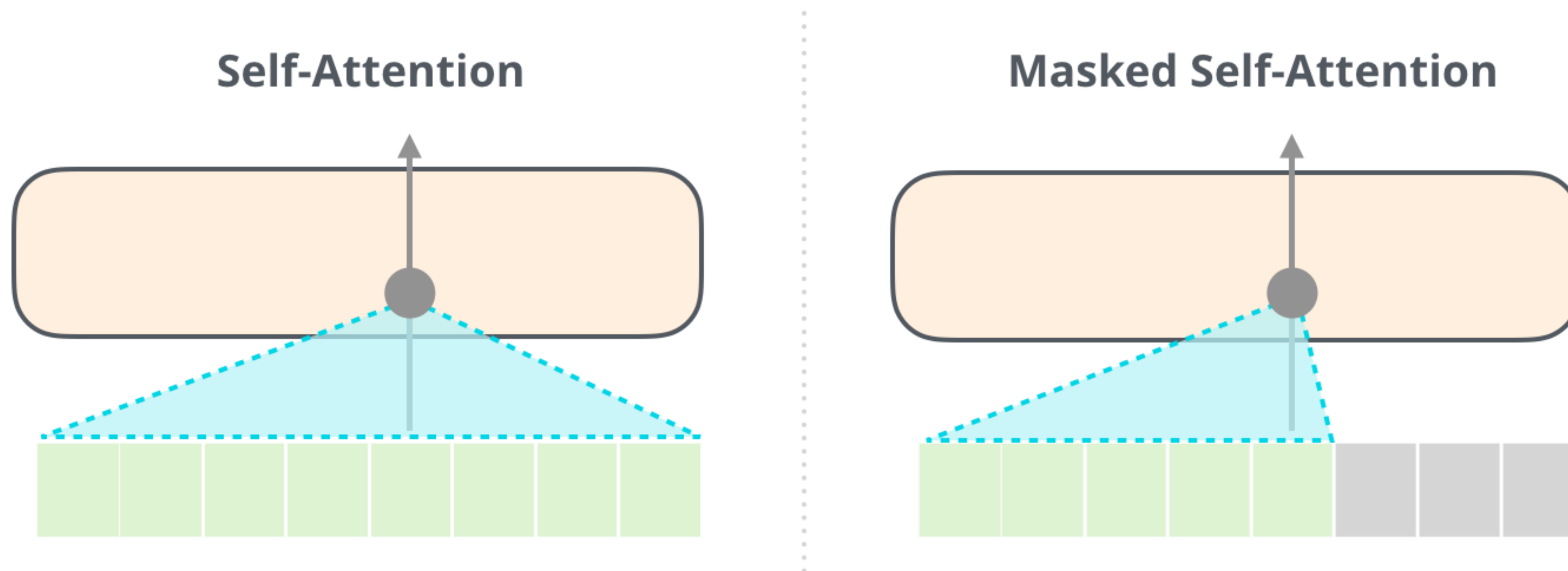
Recap from RNNs

- Bi-directionality is important in language representations → Bi-directional RNNs!
- For text generation, you can't use bidirectionally.



Masked (casual) self-attention

- Key: You can't see the future text for generation!



- Solution: for every query $\mathbf{q}_i$, only attend to $\{(\mathbf{k}_j, \mathbf{v}_j)\}, j \leq i$

How to implement this? Masking!

Masked (casual) self-attention

Queries

robot	must	obey	orders
-------	------	------	--------

Keys

robot	must	obey	orders
robot	must	obey	orders
robot	must	obey	orders
robot	must	obey	orders

Scores
(before softmax)

0.11	0.00	0.81	0.79
0.19	0.50	0.30	0.48
0.53	0.98	0.95	0.14
0.81	0.86	0.38	0.90

$\times$ $=$

Masked (casual) self-attention

Scores
(before softmax)

0.11	0.00	0.81	0.79
0.19	0.50	0.30	0.48
0.53	0.98	0.95	0.14
0.81	0.86	0.38	0.90

**Apply Attention
Mask**



Masked Scores
(before softmax)

0.11	-inf	-inf	-inf
0.19	0.50	-inf	-inf
0.53	0.98	0.95	-inf
0.81	0.86	0.38	0.90

Masked (casual) self-attention

Masked Scores
(before softmax)

0.11	-inf	-inf	-inf
0.19	0.50	-inf	-inf
0.53	0.98	0.95	-inf
0.81	0.86	0.38	0.90

Softmax
(along rows)



Scores

1	0	0	0
0.48	0.52	0	0
0.31	0.35	0.34	0
0.25	0.26	0.23	0.26

Transformers: Roadmap

1. Attention in Transformers

- ▶ From attention to self-attention
- ▶ From self-attention to multi-head self-attention
- ▶ From multi-head self-attention to masked multi-head self-attention

2. Zooming out: Transformers

- ▶ Embeddings and positional encoding
- ▶ Feedforward layers
- ▶ Residual connections + layer normalization
- ▶ Transformer encoder vs Transformer decoder vs. Transformer encoder-decoder

3. Modern variants of Transformers

- ▶ Embeddings and positional encoding
- ▶ Feedforward layers
- ▶ Pre-norm vs. Post-norm
- ▶ Layernorm vs. RMSNorm
- ▶ Activations
- ▶ Positional Encodings
- ▶ Attention